

# Final Document

---

Project Name: Dreamscape

Source Code: [GitHub](#)

Demo Video: [Google Drive](#)

Member: 111590012 林品緯 111590038 卓柏辰

## Frontend

---

### AuthView.swift

- Log In : 輸入郵箱、密碼是否正確
- Sign In : 輸入郵箱、密碼進行帳號註冊

### MainTabView.swift

- 管理 TabView 顯示畫面 : 有 HomeView、SearchView、CommunityView與SettingView。
- 按下後會切換到該頁面。

### HomewView.swift

- 主要顯示使用者當日的第一則貼文，與連結建立貼文的頁面按鈕 (Create)
- 貼文會顯示圖片、夢境描述、情緒與主題

### CreateImage.swift

這就是個包含建立貼文 (圖片) 的頁面

- 一開始使用者需要輸入夢境描述，按下 Create 會進行主題與情緒分析
- 分析完後，會將結果顯示出來，並且顯示預設圖片。
- 使用者可以點擊預設圖片，來自行上傳想要代表夢境的圖片。

### SearchView.swift

搜尋方式分為兩種：「關鍵字(Key word)」與「範圍時間(range datetime)」

- 使用者輸入關鍵字後，會根據所有貼文 title 找到最接近的貼文。若使用者未輸入關鍵字，但卻按下搜尋，則會出現所有的貼文。
- 使用者可以透過兩個滾輪來選取起訖時間，然後將這個範圍時間內的貼文顯示出來。（預設為一週）

### Community.swift

社群分為兩個部分：「熱門與分類」與「探索」。

- 熱門與分類：會有“熱門貼文排行榜”與“主題探索”
  - 排行榜會把按讚數前五名的貼文顯示出來，點擊後，會進入到該貼文的詳細頁面 (PostDetail)。
  - 主題探索為顯示前五多次的主題顯示出來，點擊後，會直接導入到“探索”，並且 filter 出按下的出題文章。
- 探索：若不是透過主題探索，會顯示所有的貼文。

## Setting.swift

Setting 包含「個人密碼更改」、「My/Saved Dreams」與「Log In/Out」的畫面

- 「My Dreams」與「Saved Dreams」會有所有跟使用者建立或收藏的所有貼文。
- 「更改密碼」中有輸入“舊密碼”、“新密碼”與“確認新密碼”。每一個輸入欄都可以跟其他 App 密碼輸入一樣，可以選擇是否要顯示出來，不然就不是預設一個點點。

## PostDetail.swift

貼文的詳細頁面，會顯示以下內容與功能：

- Dreams Title、Dream description、Dream image、Dream emotions/topics、likes（按讚數）、comments（留言數量與所有留言顯示）
- 使用者可以點擊畫面上 like 的愛心，會更新貼文按讚數。
- 使用者可以點擊留言符號，會彈出輸入留言框，輸入留言後便可以送出並更新。

## AnimatedStarfieldBackground.swift

這就是一個會一閃一閃亮晶晶，滿天都是小星星的背景。

這部分就是每一頁的背景，顏色為漸層的紫色，並且會有很多一點一點的星星在閃爍並緩慢移動。

這是認真會動的動態背景，只是上台報告的時候沒有提到，畢竟跟主要功能不太相關。

# Backend

---

## Firebase

users

```
avatar: "https://i.ibb.co/KcZFL577/image.jpg"
createdAt: 2025年6月14日 午夜12:00:00 [UTC+8]
▼ likedArticles
  0 "k0ZilW89INAsncqwPQiL"
  name: "測試帳號"
▼ savedArticles
  0 "k0ZilW89INAsncqwPQiL"
uid: "qKyFgLmv6hUa9Nkb8e62DWafpZ53"
```

- 用戶
  - avart 頭像
  - createdAt 建立時間
  - likedArticles(陣列) 喜歡文章
  - name 用戶姓名
  - savedArticles(陣列) 保存文章
  - uid 用戶 id

## articles

```
authorUid: "pP4yzbG3B5M07qmXUPsoq8fX5002"
▼ emotions
  0 "平靜"
  1 "和平"
id: ""
image: "https://i.ibb.co/hFJC84Wq/image.jpg"
likedCount: 859
savedCount: 0
text: "我站在一片金黃色的草原上，陽光柔和得像水一樣灑下來。遠方是一棵漂浮在空中的大樹，樹枝垂落下星光般的花朵。風吹過時，花瓣會發出清脆的鈴聲。我坐在樹下，一位早已過世的親人輕輕走來，沒有說話，只是靜靜地牽起我的手。我感覺所有的遺憾都慢慢融化在風裡。"
time: 2025年6月17日 上午10:10:32 [UTC+8]
title: "風裡的再見"
▼ topics
  0 "家人"
  1 "和解"
visible: true
```

- 文章
  - authorUid 作者 id
  - emotions(陣列) 情緒標籤
  - id 文章 id
  - image 文章圖片
  - likedCount 喜歡數量
  - savedCount 收藏數量
  - text 內文
  - time 建立時間
  - title 標題
  - topics(陣列) 主題標籤
  - visible 是否公開

## comments

```
articleId: "jMarvMBLVVSXdhUM6jl1"
text: "留言內容"
time: 2025年6月15日 午夜12:00:00 [UTC+8]
uid: "qKyFgLmv6hUa9Nkb8e62DWafpZ53"
```

- 留言
  - articleId 文章 id
  - text 留言內文
  - time 留言時間
  - uid 留言用戶 id

## settings

```
imgbb: "75489085225038edbc8e1
openrouter_ai: "sk-or-v1-
52e0e3357e0b0
```

- 設定
  - imgbb 圖片上傳網站 api key
  - openrouter\_ai 大型語言模型 api key

## FirebaseServivce.swift

### auth 認證相關 function

```

/// 註冊新用戶
/// - Parameters:
///   - email: 用戶電子郵件
///   - password: 用戶密碼
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func userRegister(
    mail: String,
    password: String,
    completion: @escaping (Bool, Error?) -> Void
)

```

```

/// 用戶登入
/// - Parameters:
///   - mail: 用戶電子郵件
///   - password: 用戶密碼
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func userLogin(
    mail: String,
    password: String,
    completion: @escaping (Bool, Error?) -> Void
)

```

```

/// 用戶登出
/// - Parameter completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func userLogout(completion: @escaping (Bool, Error?) -> Void)

```

```

/// 確認目前是否有用戶登入
/// - Returns: 若有用戶登入則回傳 true, 否則回傳 false
static func isUserLoggedIn() -> Bool

```

```

/// 變更目前登入用戶的密碼
/// - Parameters:
///   - newPassword: 新的密碼
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func changePassword(
    newPassword: String,
    completion: @escaping (Bool, Error?) -> Void
)

```

## user 用戶相關 function

```

/// 取得單一用戶資訊
/// - Parameters:
///   - uid: 用戶唯一識別碼
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息, 用戶資料)
static func fetchSingleUser(
    uid: String,
    completion: @escaping (Bool, Error?, User?) -> Void
)

```

```

/// 更新用戶資訊
/// - Parameters:
///   - user: 更新後的用戶物件
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func updateUser(
    user: User,
    completion: @escaping (Bool, Error?) -> Void
)

```

```

/// 建立用戶資料 (用於註冊後儲存至 Firestore)
/// - Parameters:
///   - user: 要建立的用戶物件
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func createUser(
    user: User,
    completion: @escaping (Bool, Error?) -> Void
)

```

## article 文章相關 function

```

/// 取得單一文章內容
/// - Parameters:
///   - articleId: 文章唯一識別碼
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息, 文章資料)
static func fetchSingleArticle(
    articleId: String,
    completion: @escaping (Bool, Error?, Article?) -> Void
)

```

```

/// 取得當日所有文章 (依照發布日期過濾)
/// - Parameter completion: 完成處理後回傳 (成功與否, 錯誤訊息, 文章陣列)
static func fetchTodayArticles(
    completion: @escaping (Bool, Error?, [Article]?) -> Void
)

```

```

/// 取得特定作者的所有文章
/// - Parameters:
///   - authorUid: 作者的用戶唯一識別碼
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息, 文章陣列)
static func fetchAuthorAllArticles(
    authorUid: String,
    completion: @escaping (Bool, Error?, [Article]?) -> Void
)

```

```

/// 建立新文章 (發佈文章)
/// - Parameters:
///   - article: 要建立的文章物件
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func createArticle(
    article: Article,

```

```
        completion: @escaping (Bool, Error?) -> Void
    )
```

```
/// 更新文章內容
/// - Parameters:
///   - article: 已修改的文章物件
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func updateArticle(
    article: Article,
    completion: @escaping (Bool, Error?) -> Void
)
```

```
/// 搜尋文章 (可依關鍵字與時間範圍)
/// - Parameters:
///   - keyword: 搜尋關鍵字 (可為 nil)
///   - startDate: 起始日期 (可為 nil)
///   - endDate: 結束日期 (可為 nil)
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息, 符合條件的文章陣列)
static func searchArticles(
    keyword: String?,
    startDate: Date?,
    endDate: Date?,
    completion: @escaping (Bool, Error?, [Article]?) -> Void
)
```

```
/// 按讚或取消按讚文章
/// - Parameters:
///   - like: true 為按讚, false 為取消按讚
///   - articleId: 要操作的文章 ID
///   - userId: 進行操作的用戶 ID
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func likeArticle(
    like: Bool,
    articleId: String,
    userId: String,
    completion: @escaping (Bool, Error?) -> Void
)
```

## comment 留言相關 function

```
/// 新增留言至指定文章
/// - Parameters:
///   - comment: 要建立的留言物件
///   - completion: 完成處理後回傳 (成功與否, 錯誤訊息)
static func createComment(
    comment: Comment,
    completion: @escaping (Bool, Error?) -> Void
)
```

```
/// 取得指定文章的所有留言
/// - Parameters:
```

```

/// - articleId: 文章的唯一識別碼
/// - completion: 完成處理後回傳 (成功與否, 錯誤訊息, 留言陣列)
static func fetchComments(
    for articleId: String,
    completion: @escaping (Bool, Error?, [Comment]?) -> Void
)

```

## other 其他 function

```

/// 取得指定名稱的 API 金鑰 (從 Firestore 中存取)
/// - Parameter keyName: 欲取得的金鑰名稱
/// - Returns: 金鑰字串 (若成功)
/// - Throws: 發生錯誤時會拋出例外
static func getAPIKey(for keyName: String) async throws -> String

```

```

/// 上傳圖片至伺服器或雲端儲存空間 (這裡上傳到 imgbb)
/// - Parameter image: 要上傳的 UIImage 物件
static func uploadImage(
    image: UIImage
)

```

```

/// 向 OpenRouter 發送訊息並取得回應結果
/// - Parameter text: 使用者輸入的文字
/// - Returns: 回傳兩個字串陣列: 第一個是情緒辨識結果, 第二個是主題分類結果
/// - Throws: 發生錯誤時會拋出例外
static func askOpenRouter(text: String) async throws -> ([String], [String])

```